# LAB 05: LOOPS

## CONTENT

Objective....2
CORE Guidelines & Naming conventions....2
naming conventions....2
Core guidelines....2
Exercises....3
Ex01: explore loops - for, while, do while....3
general....3
Ex02: Awaits Human Review....6
Ex03: RandomRectangles....6
general....6
description....6
Ex04: polygon....7
general....7
decagon....7
Ellipse, drawn as a many-sided polygon....8
Ex05: LoopedDrawings....8
general....8
Ex06: ConcentricLines....9
general....9
Ex07: Coding Craftmanship Challenge....10
Instructions....10
Wall of Fame....10
Submission instructions....10
Before zipping....10
Submit....11
References....11
Input/output manipulators....11
general input / output manipulators....11
Set field width....11
Set the decimal precision....11
Use fixed floating-point notation....11
Enumeration....11
ASCII table....11

Algorithm to draw circles and ellipses.....11

# OBJECTIVE

At the end of these exercises, you should be able to:

- Use the 3 loop statements : `for`, `while` and `do while`

We advise you to **make your own summary of topics** which are new to you.

# CORE GUIDELINES & NAMING CONVENTIONS

## NAMING CONVENTIONS

The **naming conventions** used in Prog1 and Prog2 promote a consistent coding style and should be followed, just as you will need to follow the conventions of any project, team, or company you end up with.

| Category | Naming Convention / Pattern | Code Example |
|---|---|---|
| **Local Variables** | **l**ower**C**amel**C**asing | `float width{};`<br>`const float innerSize{ 10.f };` |
| **Global Variables** | **g_**UpperCamelCasing | `int g_NumberOfEnemies{};`<br>`const float g_WindowWidth{ 500.0f };` |
| **Boolean Identifiers** | Verb indicating boolean state (Is..., Has…) | `bool isReady{};`<br>`bool g_IsAlive{};`<br>`bool IsKeyPressed();` |
| **Functions / Methods** | **U**pper**C**amel**C**asing (Verb-based) | `ClearBackground(); // return type void` |
| **Types (Struct, Enum)** | **U**pper**C**amel**C**asing | `struct Circle{…};`<br>`enum class LightState{…};` |
| **Enum *Values*** | **l**ower**C**amel**C**asing | `enum class Direction {none, leftUp, rightDown};` |

## CORE GUIDELINES

Always check the **PROG1: Naming Conventions & C++ Core Guidelines** document for a detailed explanation about the guidelines below.
(**New guidelines** introduced in this lab are indicated in **blue**)

- ✓ **F1;** Name meaningful operations
- ✓ **F3;** Keep functions short and simple
- ✓ **Enum.1;** Prefer enumerations over macros
- ✓ **Enum.3;** Prefer class enums over plain enums
- ✓ **Enum.5;** Don't use ALL_CAPS for enumerators
- ✓ **ES.5;** Keep scopes small → In the online guidelines, **go to this rule** and look at first code example!
- ✓ **ES.10;** Declare one name (only) per declaration
- ✓ **ES.20;** Always initialize an object
- ✓ **ES.23;** Prefer the {} initializer syntax
- ✓ **ES.48;** Avoid casts if possible
- ✓ **ES.72;** Prefer a for-statement to a while-statement when there is an obvious loop variable
- ✓ **ES.73;** Prefer a while-statement to a for-statement when there is no obvious loop variable
- ✓ **ES.74**; **ES.6;** Prefer to declare a loop variable in the initializer part of a for-statement
- ✓ **ES.75;** Avoid do-statements
- ✓ **ES.86;** Avoid modifying loop control variables inside the body of raw for-loops

- ✓ **ES.87;** Don't add redundant == or != to conditions → also **applies to conditions in loops**!
- ✓ **CON.1;** By default, make objects immutable
- ✓ **NL.1;** Don't say in comments what the code already says
- ✓ **NL.3;** Keep comments crisp
- ✓ **NL.4;** Maintain a consistent indentation style
- ✓ **NL.5;** Don't encode type information in names
- ✓ **NL.8;** Use a consistent naming style
- ✓ **NL.9;** Use ALL_CAPS only for macros
- ✓ **NL.20;** Don't place two statements on the same line
- ✓ **NR.1;** Don't force all declarations to the top

## EXERCISES

Each lab you should **create a folder** with the correct name:

- **L05_ Name_FirstName**
  - Replace **Name** and **FirstName** by your personal last and first name
    - Avoid any special characters like ` or - by replacing them with an underscore _
  - Replace **L02** with the correct lab number each week
  - All projects from this lab should be created inside this folder

### EX01: EXPLORE LOOPS - FOR, WHILE, DO WHILE

#### GENERAL

- Create a new **PROG1 CONSOLE PROJECT** called **EX01_ExploreLoops** in this lab's folder
- Topics needed for this exercise: `for`, `while` and `do while`
- Create a **function** with a clear name for every exercise below!

#### EVEN NUMBERS

Print the **even** numbers from **2 until 24** using:

1. Iteration statement **`for`**
2. Iteration statement **`while`**
3. Iteration statement **`do while`**

```
-- Even numbers --
2 4 6 8 10 12 14 16 18 20 22 24
2 4 6 8 10 12 14 16 18 20 22 24
2 4 6 8 10 12 14 16 18 20 22 24
```

#### MOVIE RATINGS

This exercise keeps asking for movie ratings until the user decides to stop. After that, they print the average rating that was given by the user.

- Ask for a movie name.
  - **Note:** when using std::cin for a string, only the part up until the first space will be placed in the variable (all the rest is seen as next input). Solve this by using the getline function: `std::getline(std::cin, xx); // replace xx with string variable`
  - **Note 2:** the getline function is part of the string library; do not forget to include it
- Then, ask for the rating they would give it [0-5]. Assume correct user input.
- ⇨ Print movie name and rating. **Use a for loop** to visualize the rating as can be seen in the screenshot.

```
-- Movie ratings --
Enter the movie you wish to rate (or type "exit" to stop): Inception
-> How many stars for this movie [0-5]? 5
| Inception | ***** |

Enter the movie you wish to rate (or type "exit" to stop): Plan 9 from Outer Space
-> How many stars for this movie [0-5]? 1
| Plan 9 from Outer Space | *---- |

Enter the movie you wish to rate (or type "exit" to stop): The Grand Budapest Hotel
-> How many stars for this movie [0-5]? 4
| The Grand Budapest Hotel | ****- |

Enter the movie you wish to rate (or type "exit" to stop): exit

Thank you for using our rating system.
-> On average, you give a rating of 3.33333 out of 5.
```

- Keep asking until the user enters the word “exit”
  - Use `std::cin.ignore();` after asking for the rating to clear the enter used for rating input
- After they typed “exit”, the average of all entered ratings is displayed (see last 2 lines in screenshot).

## RANDOM GUESS

In this exercise you’ll count how many times the rand() function needs to be called before resulting in a requested value.

- Make the user enter a number.
- Using a loop, generate random numbers until getting a number equals the entered number.
- Display how many guesses (rand calls) were needed.

```
-- Guess number --
Number to guess ? 20
Number found after 5556 guesses
```

## RANDOM GUESS – CHECK USER INPUT

Always **TEST** your code for all cases:

- e.g. enter a negative number. What do you notice?
- Stop the application using the “Stop” button of Visual Studio 

Let’s make this slightly more fool proof. Adapt the code:

- don’t start the statistics when an invalid number – not in [0, RAND_MAX] - is entered.
- RAND_MAX has the value 0x7FFF or 32767. When this is the case, give the user an appropriate message and ask again to ENTER a number.
- Repeat this until a correct value is entered.

```
-- Guess number --
Number to guess ? -6
This is a wrong number, number to guess ? -20
This is a wrong number, number to guess ? 21
Number found after 1881 guesses
```

## TRIGONOMETRY

Print a table with 4 columns: **degrees**, **radians**, **cosine**, **sine**, for angles from **0 to 180 degrees**, in **steps of 10**.

**Formatting**

By using input / output manipulators in std::cout, we can achieve the outlined result in the screenshot:

- To use input / output manipulators: `#include <iomanip>`
- Column width of 10: `std::setw(10)`
- 2 decimals of precision: `std::setprecision(2)`
- No exponents (e.g. avoid 1.23e+01): `std::fixed`
- ⇨ Figure out where in your std::cout chain each one belongs, and whether it needs to be repeated for *every* value or only needs setting once for the whole table.
- ⇨ For more information see also bottom of this page → references

```
-- Trigonometry --
   Degrees    Radians        Cos        Sin
         0       0.00       1.00       0.00
        10       0.17       0.98       0.17
        20       0.35       0.94       0.34
        30       0.52       0.87       0.50
        40       0.70       0.77       0.64
        50       0.87       0.64       0.77
        60       1.05       0.50       0.87
        70       1.22       0.34       0.94
        80       1.40       0.17       0.98
        90       1.57       0.00       1.00
       100       1.75      -0.17       0.98
       110       1.92      -0.34       0.94
       120       2.09      -0.50       0.87
       130       2.27      -0.64       0.77
       140       2.44      -0.77       0.64
       150       2.62      -0.87       0.50
       160       2.79      -0.94       0.34
       170       2.97      -0.98       0.17
       180       3.14      -1.00       0.00
```

## ASCII TABLE

Print all the letters of the alphabet and its corresponding ASCII value in 4 columns, **without using hardcoded ASCII values** however using a loop of course.

**Hint:** your loop counter doesn't have to be an int; a `char` works as a loop variable too, and 'a' or 'A' is more readable than the ASCII value number.

For more information see ASCII table

```
-- ASCII table --
a      97   n     110
b      98   o     111
c      99   p     112
d     100   q     113
e     101   r     114
f     102   s     115
g     103   t     116
h     104   u     117
i     105   v     118
j     106   w     119
k     107   x     120
l     108   y     121
m     109   z     122
```

## EX02: AWAITS HUMAN REVIEW

Unzip the **given cmake project** with the same name as the title above.

You have been given **a piece of AI-generated C++ code**. Your task is to give it a code review and identify mistakes, bad practices, guideline violations, and logic errors.

**Your goal**

- Use the Core guidelines & naming conventions section at the start of this document as reference.
  - The new ones introduced in this lab are indicated in **blue**.
- For every guideline violation, **mention the corresponding guideline code** (e.g. ES.23, NL.5).
- **Write your findings as comments** above or next to the relevant line(s) of code.
- Always **explain in short** why the code is problematic, not just what is wrong.
- For each issue you find, comment out original code and **write the corrected version underneath it**.

***IMPORTANT***

*IF YOU ARE PLANNING TO ASK AN AI TO REVIEW THE AI-GENERATED CODE FOR YOU, PLEASE SKIP THIS ASSIGNMENT ENTIRELY. YOU'LL SAVE:*

- *YOUR TIME,*
- *OUR TIME,*
- *A FEW SERVERS,*
- *AND POSSIBLY AN ENTIRE FOREST WORTH OF ELECTRICITY.*

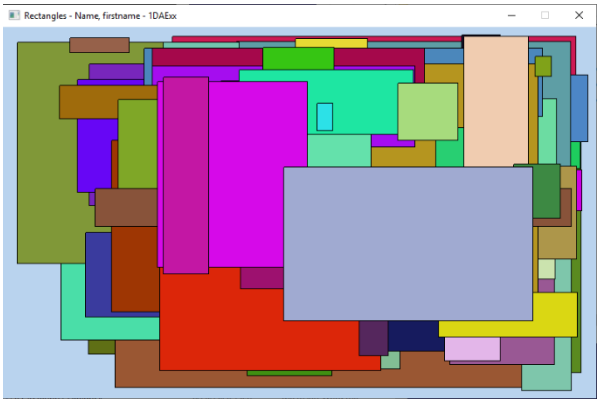
**More importantly, you'll miss the actual learning goal: developing the ability of critical thinking.** In a few weeks, you won't be graded on your ability to ask an AI for answers. You'll be graded on your ability to review code yourself. This will be done in a **closed-book** environment, without AI, notes, or online resources.

## EX03: RANDOMRECTANGLES

### GENERAL

- Create a new **PROG1 GAME PROJECT** called **RandomRectangles** in this lab's folder
- Don't forget to set the title. Change the window size into 800 x 500.

### DESCRIPTION




In **LAB 02** we randomly generated a random rectangle **each frame**, making sure it kept a small margin on each side of the window.

- **Copy** the solution of **EX06_RandomRects** to this week's lab folder
- Remove the **out** and **.vs** folder before opening

The existing code generates 1 single rectangle each frame. This means we have to wait 100 frames (at 60 frames per second by default) for 100 rectangles.

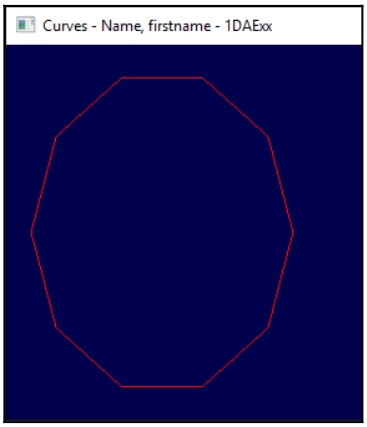
- **Change** the code so that ***each* frame *100* rectangles** are generated at once (using a loop, obviously).

- This goes incredibly fast. To **test**, **slow down the animation**, using `Sleep(500);` *outside* your loop. *(PS: Sleep is **not** ISO C++ and freezes your entire window, which we will want to avoid during a game!! We only introduce this for testing purposes. Do **not** make a habit of using it.)*

## EX04: POLYGON

### GENERAL

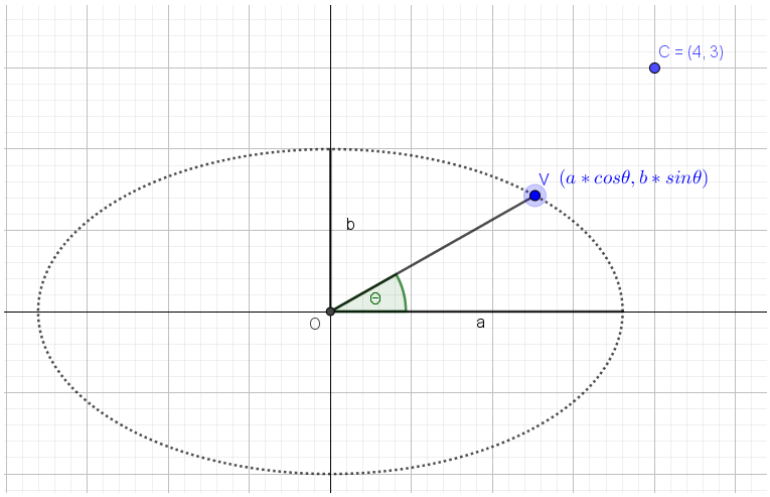
- Create a new **PROG1 GAME PROJECT** called **EX04_Polygon** in this lab's folder
- Don't forget to set the title.
- Change the window size to 500 x 600.




In this project you will draw a red decagon (a 10-sided polygon) sitting on the outline of an ellipse. We will **not** use the engine's `DrawPolygon` function; just all **separate lines**.

More information about this exercise follows. In the end you should have a window that looks like the one in the screenshot →

### DECAGON




Each corner sits on an ellipse, at angle `θ` from the center:

```
Vx = Cx + a * cos θ
Vy = Cy + b * sin θ
```

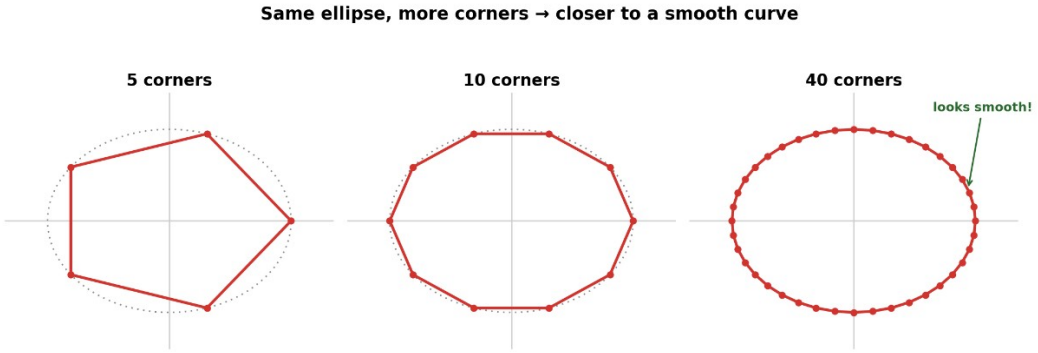
(`a`, `b` = the ellipse's radii, `C` = its center, use `cosf`/`sinf`)

Use a loop to compute the corners: split a full turn (`2π`) into equal angle steps based on how many corners you want and connect each new point to the previous one (plus one final line back to the first point, to close the shape).

### ELLIPSE, DRAWN AS A MANY-SIDED POLYGON

Hopefully you defined the number of corners as a `const` variable in the previous exercise, so it can be changed easily in one place. If you didn't, refactor your code now and replace any hardcoded corner counts with a `const` variable before continuing.

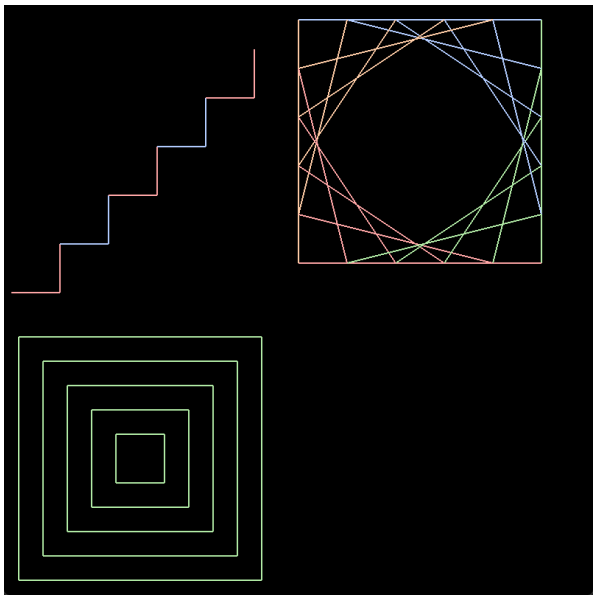
Try it yourself: start with a small number of corners so the polygon is obviously a polygon. Then bump it up - 10, then 20, then 50 - and watch it turn into what looks like a smooth ellipse. You'll see this same trick again anytime a game needs to draw circles, arcs, or curves; it is exactly what a function like the engine's `DrawEllipse` is doing internally.



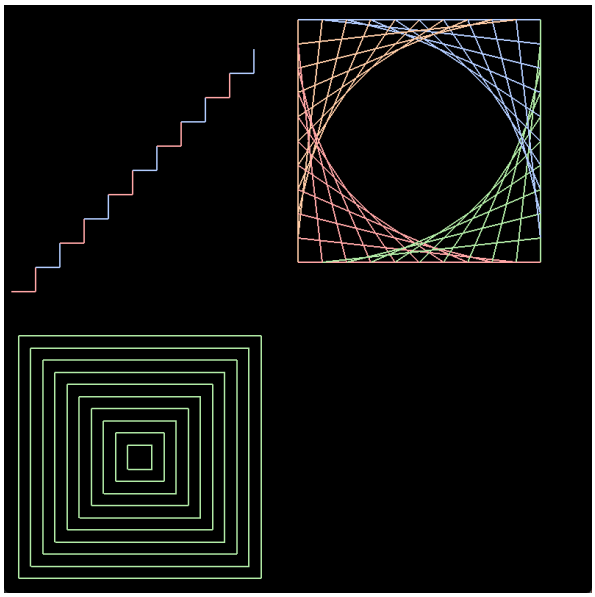

## EX05: LOOPEDDRAWINGS

### GENERAL

- Create a new **PROG1 GAME PROJECT** called **EX05_LoopedDrawings** in this lab's folder
- Don't forget to set the title. Change the window size to 800 x 800.



*Example of default **N = 5.***



*Example when **N = 10.***

### COLORS

Colors used in screenshots:

```
const Color4f skyBlue{ 0.70f, 0.80f, 1.00f, 1.f };
const Color4f salmon{ 1.00f, 0.65f, 0.65f, 1.f };
const Color4f lightGreen{ 0.70f, 0.92f, 0.65f, 1.f };
const Color4f peach{ 1.00f, 0.80f, 0.65f, 1.f };
```

### A STAIR WITH N STEPS, ALTERNATING COLORS

*Create a function for this and following shapes!*

**Given:** the *full* height of the stairs + the number of steps N.

**Figure out:** the size of each step, their position, and how to alternate the color.
**Hint:** each loop draws a set of 2 lines.

### N CONCENTRIC SQUARES

**Given:** the size of the *biggest* square, and the number of squares N.

**Figure out:** and the distance between one square and the next, the size of each square as you move inward, and how to keep them centered.

### FOUR SETS OF N SPINNING LINES

N sets of spinning lines, each set consisting of 4 lines, each with their own color.

**Given:** the bounding rectangle (position + size) and the number of sets N.

**Figure out:** N **evenly spaced points along each side**, and how to pair up a point on one side with a point on the next side so the lines fan out into that spinning shape, then reuse that same logic for all four sides..

### MAKE N ADJUSTABLE FOR ALL DRAWINGS

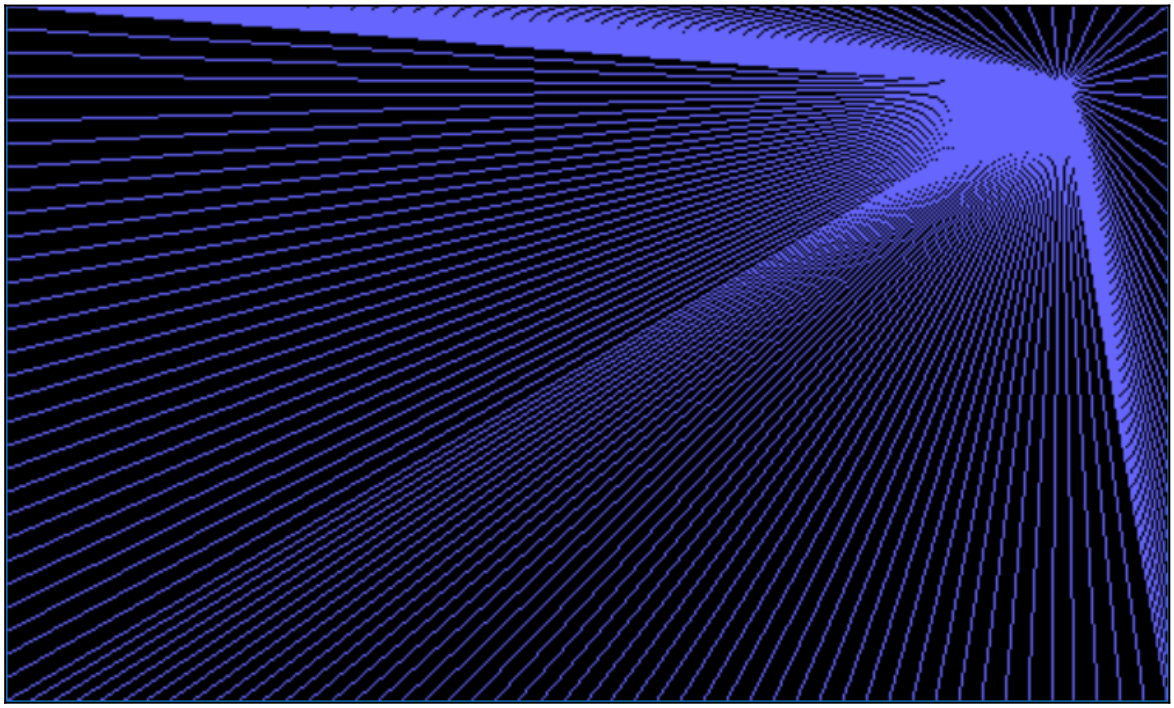
- N starts at 5 by default.
- The up/down arrow keys increase/decrease N, clamped to [4, 20].
- All three drawings update immediately to reflect the new N.
- Print the new N to the console each time it changes.

## EX06: CONCENTRICLINES

### GENERAL

- Create a new **PROG1 GAME PROJECT** called **EX06_ConcentricLines** in this lab's folder
- Don't forget to set the title.

After doing the exercises you should have a window that looks similar to this.



Define and call a `DrawConcentricLines` function that draws lines from the edges of the window to the current mouse position. There are **9 pixels between each line** on all edges.

Then **switch between 3 colors** each time the **right arrow key** is pressed:

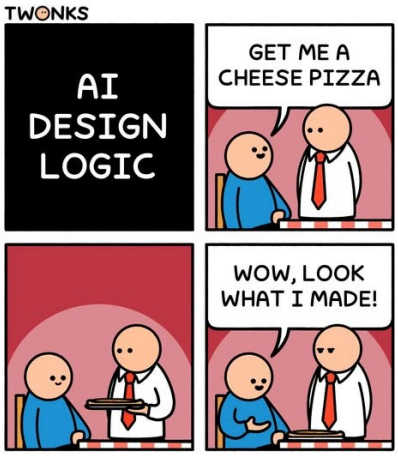
- Define a `Color` enum type with 3 enumeration values: `red`, `green` and `blue`.
- Define a global variable `g_Color` of that enum type.
- When the right arrow key is pressed, make `g_Color` switch to the next color, to that end define a function `SwitchColor`. Do not set the Draw Color in the `SwitchColor` function, its only purpose is to switch the color, **not** to set the draw color!
- In the function, check this variable to decide which color you should use to draw the lines.

## EX07: CODING CRAFTMANSHIP CHALLENGE

**DO NOT SKIP THIS EXERCISE; EXPERIMENTING WITH CODE IN A SELF-MADE PROJECT IS BY FAR THE BEST WAY TO MASTER PROGRAMMING!**

### INSTRUCTIONS

Create a **Prog1 Game Project** called **CodingCraft03** that demonstrates **animations**.




Create a **small original application** that demonstrates the concepts we've covered so far.

- ✓ Create **your own** original project.
- ✓ **Only** use **concepts that have been covered** in labs already.
- ✓ Write code you can explain yourself.
- x **No** AI-generated code.
- x **No** copied solutions from the internet.
- x **No** copying another student's work.

For example, this week: draw your favorite logo or character using basic shapes or find inspiration by searching for daily minimal in Google images.

### WALL OF FAME

Every week, some projects will earn a place on the lab's Wall of Fame.

**A project does not need to be complex to deserve a spot.** Original ideas, clever concepts, fun themes, polished outputs, and well-written code are just as valuable. Projects are shown **anonymously** so you can choose whether you reveal yourself as the creator or not.

The **goal** is to **learn, experiment, and become a better programmer**. The Wall of Fame only works if every project genuinely reflects your own work. **Let your imagination run wild and work on a programming project YOU love!**

## SUBMISSION INSTRUCTIONS

You must submit **two files**:

- A **zip file** containing your **complete lab folder** of this week with all exercises.
- **CodingCraft03.zip** containing only your Coding Craftsmanship Challenge project.

### BEFORE ZIPPING

For every project:

- **Close** Visual Studio.
- Delete the **out** folder.
- Delete the hidden **.vs** folder.

### SUBMIT

- Upload **both files** with the correct assignment before the start of next week's first lab.
- Make sure to complete the **quiz**.
- *These steps are required to unlock the next lab assignment!*

⚠️ Your submission may be discussed during a code review. Make sure you understand and can explain your own code.

## REFERENCES

### INPUT/OUTPUT MANIPULATORS

#### GENERAL INPUT / OUTPUT MANIPULATORS

https://en.cppreference.com/w/cpp/header/iomanip

#### SET FIELD WIDTH

http://www.cplusplus.com/reference/iomanip/setw/

#### SET THE DECIMAL PRECISION

http://www.cplusplus.com/reference/iomanip/setprecision/

#### USE FIXED FLOATING-POINT NOTATION

http://www.cplusplus.com/reference/ios/fixed/

### ENUMERATION

https://www.youtube.com/watch?v=Pxvvr5FAWxg

http://www.learncpp.com/cpp-tutorial/4-5a-enum-classes/

### ASCII TABLE

http://www.learncpp.com/cpp-tutorial/27-chars/

## ALGORITHM TO DRAW CIRCLES AND ELLIPSES

http://www.mathopenref.com/coordcirclealgorithm.html